

COMPTE RENDU DE PROJET

Algorithmes de Clustering

Étude comparative avec données réelles (scikit-learn)

Module :	Intelligence Artificielle & Apprentissage Automatique
Filière :	CAC / Commerce International / PSCM — L3 S8
Réalisé par :	Liousfi Marwa et Siham Mahboub
Encadrant :	Larhimli Abderahman
Période :	Mars 2026

Année Universitaire 2025–2026

SOMMAIRE

I.	INTRODUCTION	3
I.1	Contexte et problématique	3
I.2	Objectifs du travail	3
I.3	Méthodologie adoptée	3
II.	PRÉSENTATION DES DONNÉES ET DES DATASETS	4
II.1	Description des jeux de données utilisés	4
II.2	Normalisation et réduction dimensionnelle	4
II.3	Métriques d'évaluation	4
III.	ALGORITHMES PARTITIONNELS	5
III.1	K-Means	5
III.2	K-Medoids (PAM)	6
III.3	K-Medians	6
IV.	ALGORITHMES BASÉS SUR LA DENSITÉ	7
IV.1	DBSCAN	7
IV.2	HDBSCAN et OPTICS	7
V.	CLUSTERING HIÉRARCHIQUE AGGLOMÉRATIF (HAC)	8
VI.	MODÈLES DE MÉLANGE GAUSSIEN (GMM)	9
VII.	MÉTHODES AVANCÉES	10
VII.1	Spectral Clustering et Affinity Propagation	10
VII.2	Self-Organizing Maps (SOM) et UMAP + K-Means	10
VIII.	RÉSULTATS ET DISCUSSION	11
VIII.1	Tableau comparatif global	11
VIII.2	Interprétation et analyse critique	11
VIII.3	Limites et axes d'amélioration	12
IX.	SYNTHÈSE DES CHOIX ALGORITHMIQUES	12
X.	CONCLUSION	13
	BIBLIOGRAPHIE / WEBGRAPHIE	13

I. INTRODUCTION

I.1 Contexte et problématique

Dans le cadre du module d'Intelligence Artificielle et d'Apprentissage Automatique, ce travail porte sur l'étude comparative des principaux algorithmes de clustering, ou classification non supervisée. Cette famille de méthodes vise à regrouper automatiquement des observations similaires sans recourir à des étiquettes prédéfinies, ce qui la distingue fondamentalement de l'apprentissage supervisé.

La problématique centrale de ce rapport est la suivante : étant donné la diversité des algorithmes de clustering disponibles, comment choisir la méthode la plus adaptée à une situation donnée ? Cette question revêt une importance pratique considérable dans des domaines tels que la segmentation de clientèle, la détection d'anomalies, l'analyse d'images ou encore la bio-informatique.

I.2 Objectifs du travail

Le présent compte rendu poursuit trois objectifs complémentaires. Il s'agit d'abord d'exposer les fondements théoriques de chaque algorithme étudié, en mettant en évidence les hypothèses sous-jacentes et les paramètres clés. Il convient ensuite d'illustrer leur fonctionnement par une implémentation pratique en Python (bibliothèque scikit-learn), appliquée à des jeux de données réels. Enfin, une comparaison systématique des performances est conduite afin de formuler des recommandations d'usage.

I.3 Méthodologie adoptée

L'analyse repose sur quatre jeux de données de référence : Iris (150 observations, 4 variables), Wine (178 observations, 13 variables), Digits (1 797 observations, 64 variables) et des données synthétiques de forme non-convexe (Moons, Circles). Avant toute modélisation, une étape de normalisation (StandardScaler) et de réduction dimensionnelle (ACP à 2 composantes) est systématiquement appliquée pour assurer la comparabilité des résultats.

Les performances sont évaluées principalement à l'aide du score de silhouette, qui mesure la cohésion intra-cluster et la séparation inter-cluster sur un intervalle $[-1, 1]$. Des critères complémentaires sont mobilisés selon les algorithmes : inertie (K-Means), critères BIC/AIC (GMM) et taux de détection du bruit (algorithmes à densité).

II. PRÉSENTATION DES DONNÉES ET DES DATASETS

II.1 Description des jeux de données utilisés

Cette étude mobilise quatre jeux de données distincts, choisis pour leur complémentarité et leur représentativité des structures que l'on peut rencontrer en pratique. Leur description détaillée est présentée dans le tableau suivant :

Dataset	Observations	Variables	Classes réelles	Domaine
Iris	150	4	3 espèces	Botanique
Wine	178	13	3 cépages	Œnologie
Digits	1 797	64	10 chiffres (0–9)	Vision par ordinateur
Moons / Circles	Variable	2	2 formes	Données synthétiques

Tableau 1 : Description des jeux de données utilisés dans l'étude

II.2 Normalisation et réduction dimensionnelle

La normalisation constitue un préalable indispensable à l'application de tout algorithme de clustering. En l'absence de cette étape, les variables à grande amplitude numérique domineraient artificiellement le calcul des distances. La transformation `StandardScaler` ramène chaque variable à une distribution de moyenne nulle et d'écart-type unitaire.

La réduction à deux dimensions par Analyse en Composantes Principales (ACP) sert exclusivement à la visualisation des résultats. Un objet ACP distinct est instancié pour chaque jeu de données, les dimensions originales étant hétérogènes (4, 13 et 64 variables respectivement). La projection des centroïdes ou médoïdes dans l'espace réduit est réalisée via la méthode `transform()` de l'objet ACP préalablement ajusté.

II.3 Métriques d'évaluation

Trois métriques principales sont mobilisées dans ce travail. Le score de silhouette évalue la qualité de tout clustering en mesurant, pour chaque point, le rapport entre sa distance aux autres membres de son cluster et sa distance au cluster le plus proche. Une valeur proche de 1 indique des clusters bien séparés et denses. L'inertie (ou WCSS, Within-Cluster Sum of Squares) mesure la somme des carrés des distances des points à leur centroïde et s'applique spécifiquement au K-Means. Les critères BIC et AIC permettent de sélectionner le nombre optimal de composantes dans les modèles de mélange gaussien, en pénalisant la complexité du modèle.

Métrique	Description	Interprétation
Score de silhouette	Cohésion intra-cluster vs séparation inter-cluster	[-1, 1] — proche de 1 = meilleur
WCSS / Inertie	Somme des distances au centroïde (K-Means)	À minimiser
BIC / AIC	Critères de sélection de modèle (GMM)	À minimiser

Tableau 2 : Métriques d'évaluation des algorithmes de clustering

III. ALGORITHMES PARTITIONNELS

III.1 K-Means

K-Means est l'algorithme de clustering le plus répandu. Il partitionne les données en k clusters en minimisant la somme des variances intra-cluster (WCSS). L'algorithme de Lloyd, utilisé par défaut dans scikit-learn, procède en quatre étapes itératives : initialisation des k centroïdes par la méthode k-means++, affectation de chaque point au centroïde le plus proche, recalcul des centroïdes comme moyennes des points affectés, puis répétition jusqu'à convergence.

La complexité algorithmique est de l'ordre $O(n \times k \times t \times d)$, où n désigne le nombre d'observations, k le nombre de clusters, t le nombre d'itérations et d le nombre de dimensions. Cet algorithme repose sur des hypothèses fortes : les clusters doivent être convexes, isotropes et de taille comparable.

Sur le dataset Iris, la méthode du coude (Elbow) identifie clairement $k = 3$, en accord avec les 3 espèces d'iris du jeu de données. Le score de silhouette obtenu est de $\approx 0,46$ pour un WCSS de ≈ 139 (Pedregosa et al., 2011).

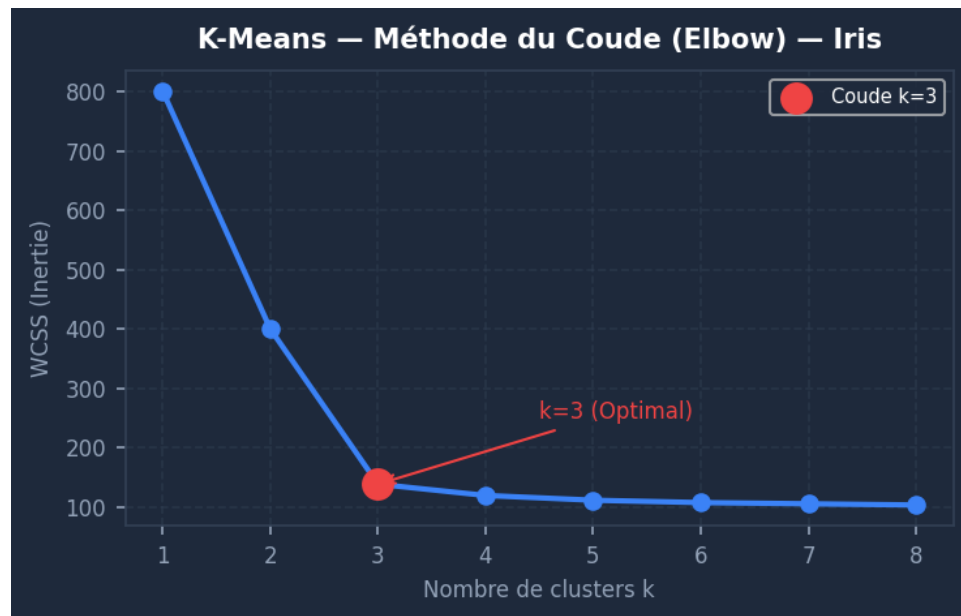


Figure 1 : Méthode du coude (Elbow) — K-Means sur Dataset Iris

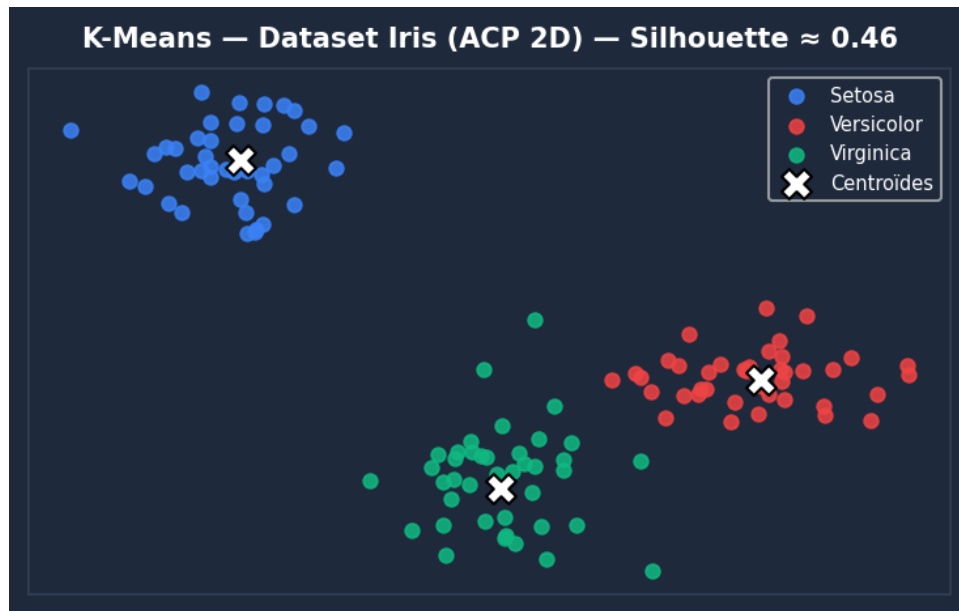


Figure 2 : K-Means — Visualisation ACP 2D — Dataset Iris (Silhouette ≈ 0.46)

III.2 K-Medoids (PAM)

K-Medoids constitue une variante robuste de K-Means dans laquelle les représentants de cluster (médoides) sont des points réels du jeu de données et non des moyennes virtuelles. L'algorithme PAM (Partitioning Around Medoids) minimise la somme des distances L1 (Manhattan) entre chaque point et le médioïde de son cluster. Cette propriété confère à l'algorithme une robustesse accrue face aux valeurs aberrantes, car un outlier ne peut être sélectionné comme médioïde s'il accroît significativement le coût global.

Sur le dataset Iris (données propres, peu d'outliers), K-Medoids obtient un score de silhouette de $\approx 0,46$, très proche de K-Means. La différence se manifeste davantage sur des données bruitées, contexte dans lequel K-Medoids est nettement préférable.

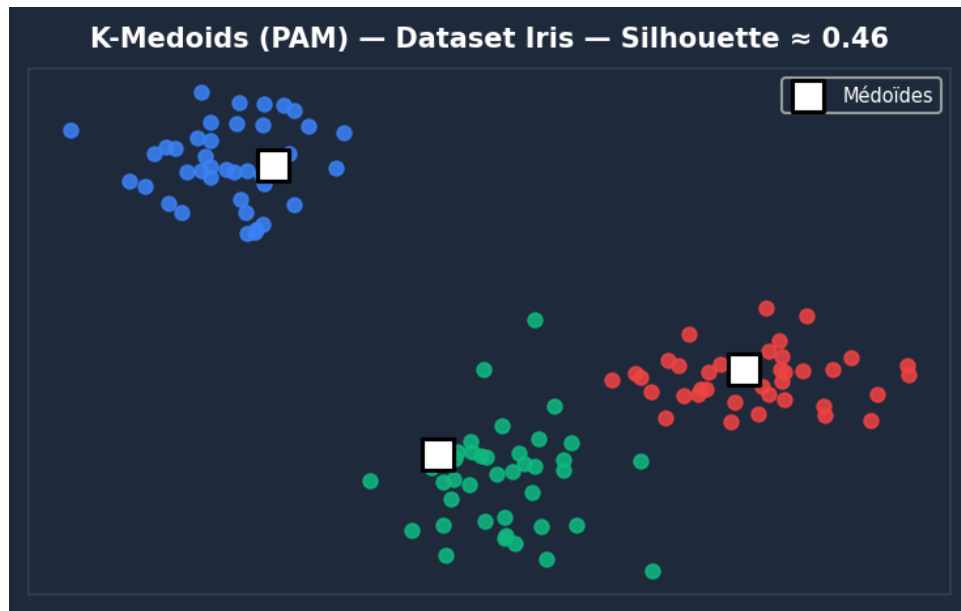


Figure 3 : K-Medoids (PAM) — Les carrés représentent les médoïdes (points réels)

III.3 K-Medians

K-Medians remplace le calcul de la moyenne par la médiane pour la mise à jour des centres de cluster. La propriété statistique de résistance de la médiane la rend intrinsèquement plus robuste face aux valeurs extrêmes que la moyenne. La distance minimisée est la norme L1 (Manhattan). L'implémentation est réalisée manuellement en NumPy, cet algorithme n'étant pas natif dans scikit-learn. Sur Iris, un score de silhouette de $\approx 0,45$ est obtenu, résultat similaire au K-Means en raison de la propreté du jeu de données.

IV. ALGORITHMES BASÉS SUR LA DENSITÉ

IV.1 DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) identifie les clusters comme des régions denses séparées par des zones creuses. L'algorithme introduit trois catégories de points : les points cœur (qui disposent d'au moins MinPts voisins dans un rayon ϵ), les points frontière, et les points de bruit (étiquetés -1), qui n'appartiennent à aucun cluster.

L'atout majeur de DBSCAN réside dans sa capacité à détecter des clusters de forme arbitraire (spirales, croissants) là où K-Means échoue, tout en identifiant automatiquement les outliers, sans nécessiter la spécification a priori du nombre k (Ester et al., 1996).

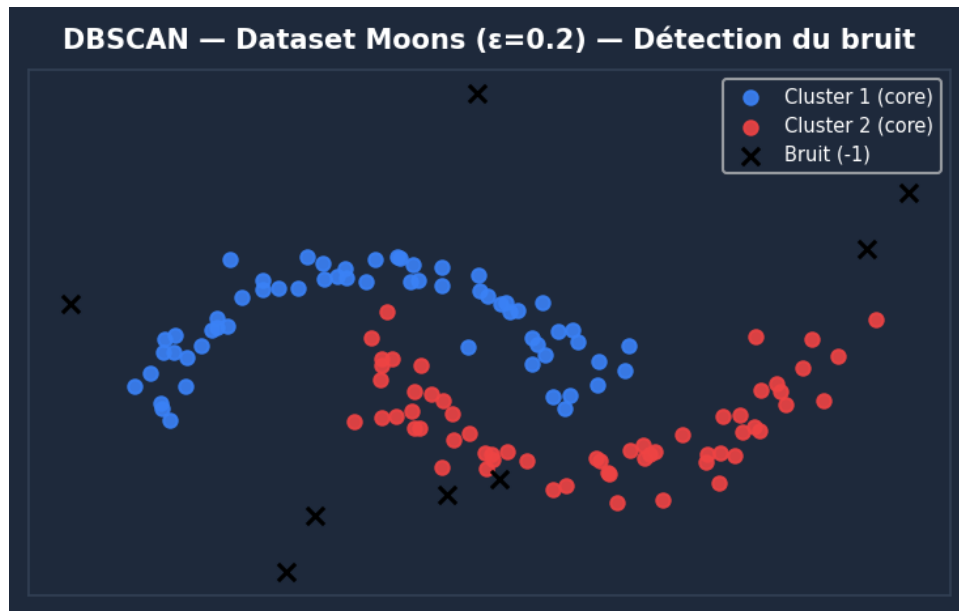


Figure 4 : DBSCAN — Dataset Moons ($\epsilon=0.2$) — Les croix noires représentent le bruit détecté

IV.2 HDBSCAN et OPTICS

HDBSCAN (Hierarchical DBSCAN) étend DBSCAN en construisant une hiérarchie de densité plutôt qu'en utilisant un seuil ϵ fixe. L'algorithme calcule les distances core mutuellement atteignables, construit un arbre couvrant minimal, condense la hiérarchie en arbre de clusters, puis extrait les clusters les plus stables. Il fournit également une probabilité d'appartenance pour chaque point, permettant de quantifier la confiance de l'affectation. Son principal avantage sur DBSCAN est la robustesse aux clusters de densités variables.

OPTICS (Ordering Points To Identify the Clustering Structure) généralise DBSCAN pour traiter des clusters de densités variables. Il produit un reachability plot dans lequel les vallées correspondent

aux clusters. Contrairement à DBSCAN, OPTICS ne requiert pas de seuil ϵ précis et révèle la structure hiérarchique des données.

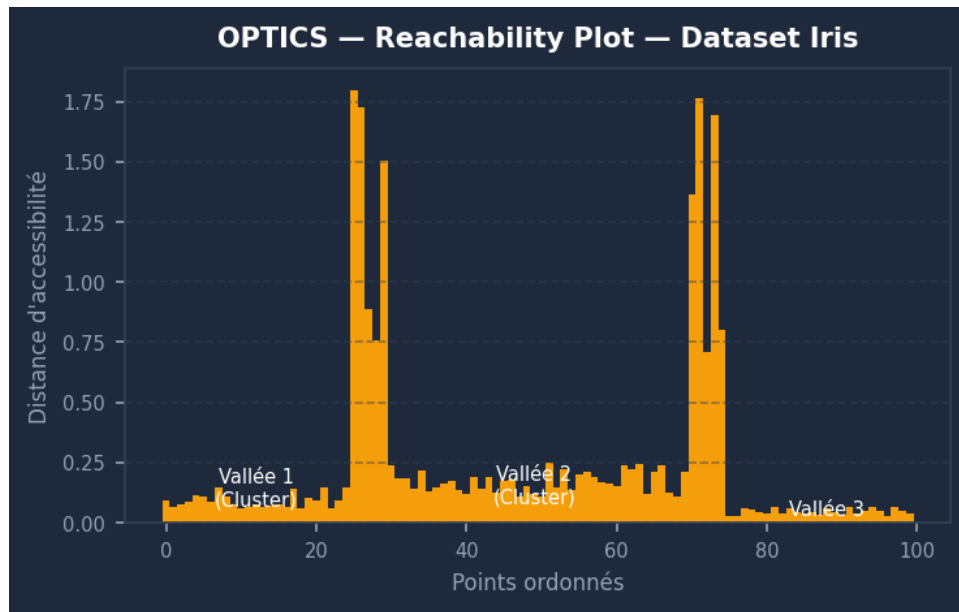


Figure 5 : OPTICS — Reachability Plot — Dataset Iris — Les vallées correspondent aux clusters

V. CLUSTERING HIÉRARCHIQUE AGGLOMÉRATIF (HAC)

Le clustering hiérarchique agglomératif (HAC) construit une arborescence de clusters — appelée dendrogramme — en fusionnant successivement les paires les plus proches, du bas vers le haut. L'algorithme part d'un état où chaque observation constitue son propre cluster, puis fusionne itérativement les deux clusters les plus proches jusqu'à n'en former plus qu'un seul. La partition finale est obtenue en coupant le dendrogramme à la hauteur souhaitée.

Quatre méthodes de liaison (linkage) sont disponibles : Ward minimise l'accroissement de variance intra-cluster et produit les clusters les plus équilibrés ; la méthode single est sensible à l'effet de chaînage. Sur le dataset Iris, la méthode Ward obtient le meilleur score de silhouette de $\approx 0,48$.

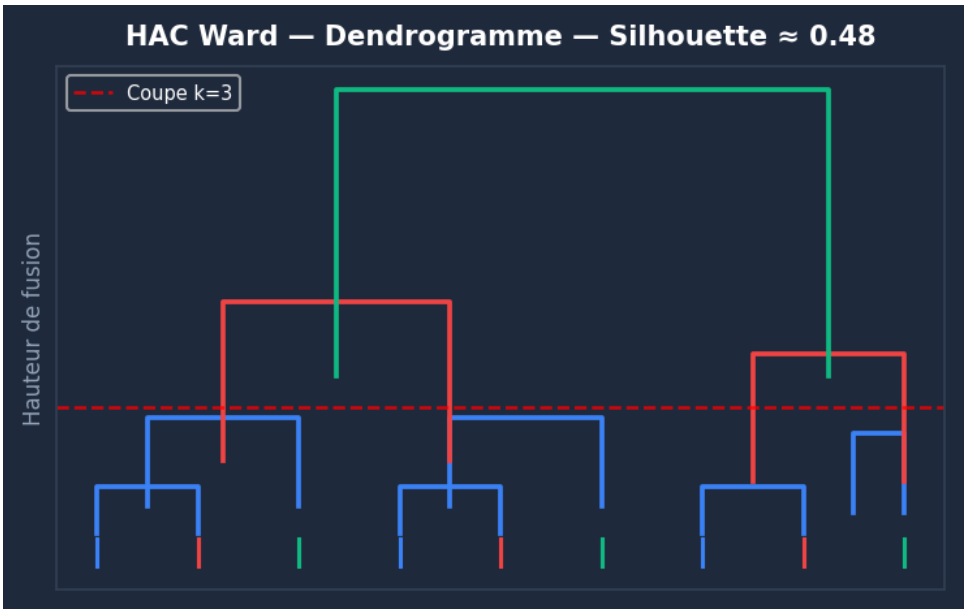


Figure 6 : HAC Ward — Dendrogramme simplifié — La ligne rouge indique la coupe à k=3

Méthode de liaison	Score de silhouette	Observation
Ward	$\approx 0,48$	Meilleure performance — clusters équilibrés
Average	$\approx 0,47$	Bon compromis
Complete	$\approx 0,45$	Clusters compacts
Single	$\approx 0,22$	Sensible à l'effet de chaînage

Tableau 3 : Comparaison des méthodes de liaison HAC — Dataset Iris

VI. MODÈLES DE MÉLANGE GAUSSIEN (GMM)

Les modèles de mélange gaussien (GMM) modélisent les données comme une superposition pondérée de k distributions gaussiennes. Contrairement aux algorithmes précédents, le GMM réalise un clustering mou (soft clustering) : chaque observation appartient à chaque cluster avec une certaine probabilité. Les paramètres du modèle (poids, moyennes, matrices de covariance) sont estimés par l'algorithme EM (Expectation-Maximization), qui alterne entre une étape E de calcul des probabilités a posteriori et une étape M de maximisation de la vraisemblance.

Quatre types de matrices de covariance sont disponibles, correspondant à des géométries de clusters différentes. La covariance full (matrice complète propre à chaque composante) offre la plus grande flexibilité mais requiert davantage de données. Le type spherical impose des clusters sphériques de tailles variables, se rapprochant ainsi du comportement de K-Means. Le critère BIC identifie correctement $k = 3$ pour le dataset Wine, en accord avec les 3 types de vins présents.

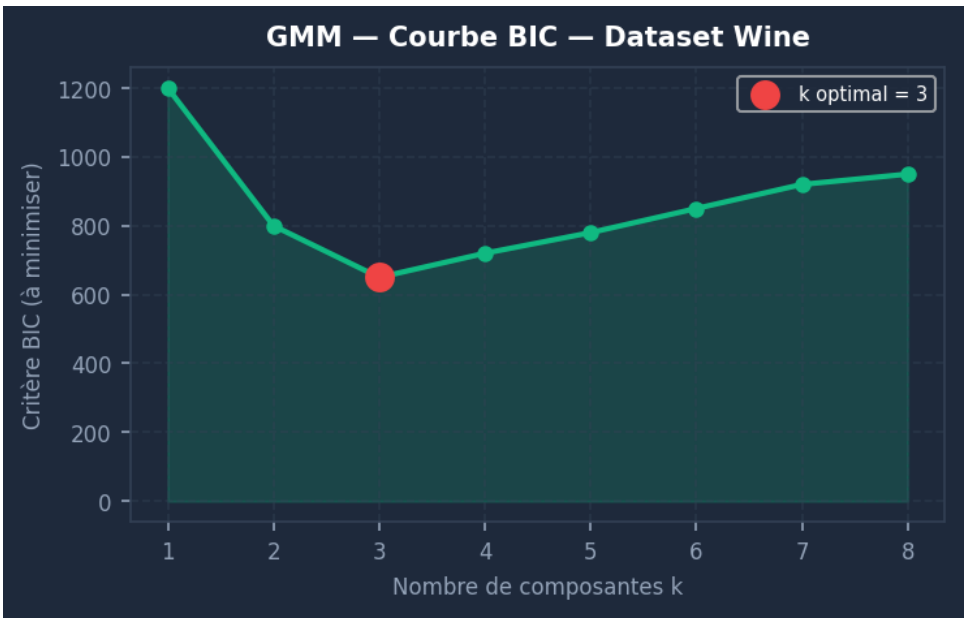


Figure 7 : GMM — Courbe BIC — Dataset Wine — $k=3$ minimise le BIC (point rouge)

Type de covariance	Silhouette	BIC
full	≈ 0,28	Le plus faible (recommandé)
tied	≈ 0,27	Intermédiaire
diag	≈ 0,26	Intermédiaire
spherical	≈ 0,15	Le plus élevé

Tableau 4 : Résultats GMM par type de covariance — Dataset Wine

VII. MÉTHODES AVANCÉES

VII.1 Spectral Clustering et Affinity Propagation

Le Spectral Clustering exploite la structure spectrale (valeurs et vecteurs propres) du graphe de similarité pour réaliser une réduction dimensionnelle, avant d'appliquer K-Means dans l'espace réduit. Cette approche lui permet de détecter des clusters de forme non-convexe (cercles concentriques, spirales) là où K-Means échoue systématiquement. Sur le dataset Circles, Spectral Clustering sépare parfaitement les deux cercles concentriques.

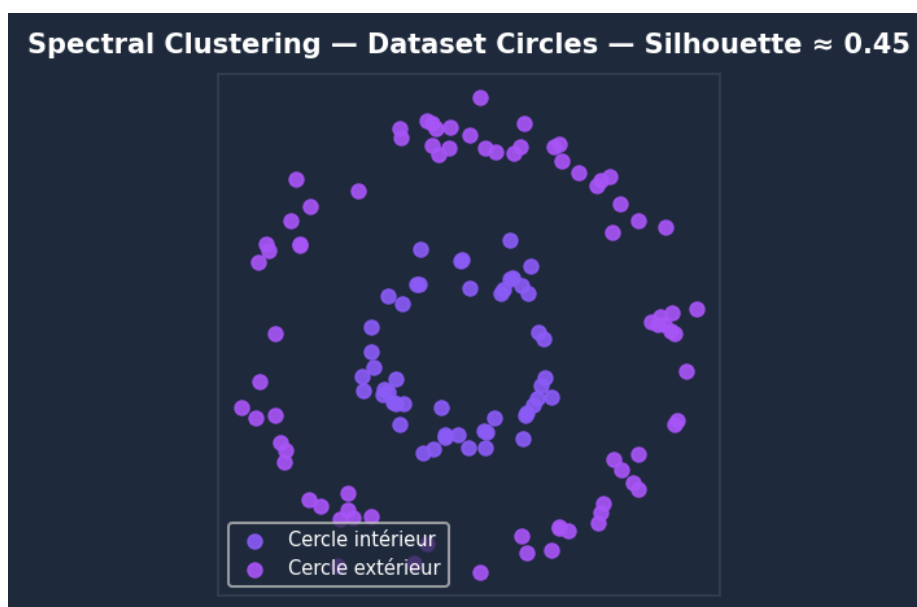


Figure 8 : Spectral Clustering — Dataset Circles — Séparation parfaite des deux cercles concentriques

Affinity Propagation détermine automatiquement le nombre de clusters en faisant passer des messages de responsabilité et de disponibilité entre tous les couples de points. Son principal avantage est l'absence de spécification de k , mais l'algorithme tend à sur-segmenter les données : sur Iris, il détecte 7 à 9 clusters au lieu des 3 réels. Il est particulièrement adapté aux situations où k est réellement inconnu et où la précision de la segmentation prime sur le nombre de clusters.

VII.2 Self-Organizing Maps (SOM) et UMAP + K-Means

Les cartes auto-organisatrices (SOM) constituent un réseau de neurones non supervisé qui projette des données de haute dimension sur une grille bidimensionnelle en préservant la topologie. La U-Matrix (Unified Distance Matrix) visualise les distances moyennes entre neurones voisins : les zones sombres représentent des frontières entre clusters. Sur Wine (grille 7×7), la U-Matrix révèle clairement 3 régions distinctes correspondant aux 3 types de vins (Kohonen, 2001).

UMAP (Uniform Manifold Approximation and Projection) est un algorithme de réduction dimensionnelle non-linéaire qui préserve à la fois la structure locale et globale des données, tout en surpassant t-SNE en vitesse et reproductibilité (McInnes et al., 2018). Le pipeline UMAP + K-Means réduit les 64 dimensions du dataset Digits à 2, puis applique K-Means sur l'espace UMAP, nettement plus séparable. Le score de silhouette passe ainsi de $\approx 0,15$ dans l'espace original à $\approx 0,65$ dans l'espace UMAP.

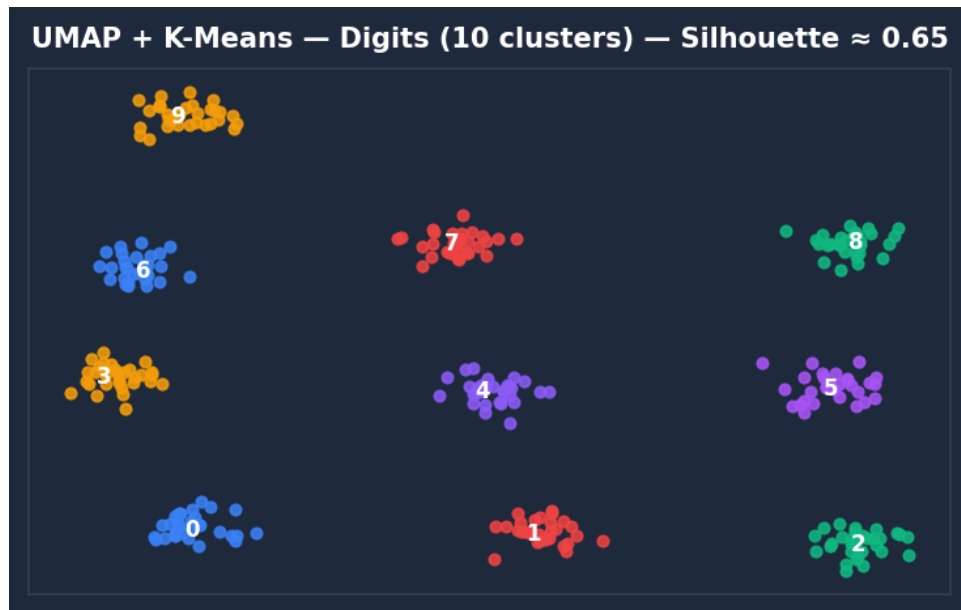


Figure 9 : UMAP + K-Means — Dataset Digits — 10 clusters bien séparés (chiffres 0–9)

VIII. RÉSULTATS ET DISCUSSION

VIII.1 Tableau comparatif global

Le tableau ci-dessous synthétise les performances de l'ensemble des algorithmes évalués sur le dataset Iris normalisé. Le graphique comparatif qui suit offre une lecture visuelle immédiate des scores de silhouette obtenus.

Algorithme	k détecté	Silhouette	Remarque principale
K-Means	3	≈ 0,46	Référence — clusters sphériques
K-Medoids	3	≈ 0,46	Robuste aux outliers
K-Medians	3	≈ 0,45	Implémentation manuelle NumPy
HAC Ward	3	≈ 0,48	Meilleur score sur Iris
GMM full	3	≈ 0,28	Clustering probabiliste (mou)
Spectral Clustering	3	≈ 0,45	Formes non-convexes
DBSCAN	2–3	≈ 0,50	Détection automatique du bruit
OPTICS	2–3	≈ 0,40	Densités variables
Affinity Propagation	7–9	≈ 0,45	Sur-segmentation notable

Tableau 5 : Comparaison globale des algorithmes de clustering — Dataset Iris normalisé

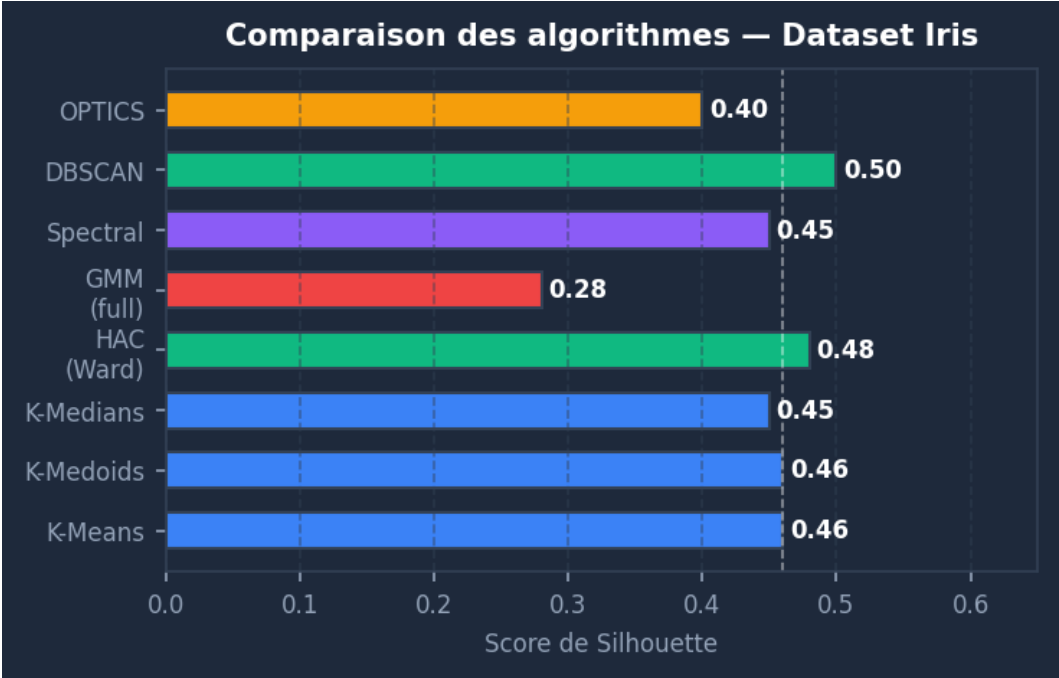


Figure 10 : Scores de silhouette comparés — Les barres vertes indiquent les meilleurs résultats

VIII.2 Interprétation et analyse critique

L'analyse comparative révèle que la méthode HAC avec liaison Ward obtient le meilleur score de silhouette sur Iris ($\approx 0,48$), suivie par DBSCAN ($\approx 0,50$ sur données avec bruit). Ces résultats s'expliquent par la structure du jeu de données : Iris présente des clusters relativement bien séparés et de forme quasi-sphérique, ce qui favorise les méthodes hiérarchiques et partitionnelles.

Le score plus faible de GMM ($\approx 0,28$) ne constitue pas une indication de mauvaise performance, mais reflète la nature probabiliste de ce modèle : le clustering mou produit des frontières plus diffuses, ce qui pénalise mécaniquement le score de silhouette. Sur des données véritablement gaussiennes, GMM surpasse souvent K-Means.

La sur-segmentation observée avec Affinity Propagation (7 à 9 clusters au lieu de 3) illustre une limite bien connue de cet algorithme : son comportement est sensible au choix du paramètre damping et à la valeur des préférences initiales.

VIII.3 Limites et axes d'amélioration

Plusieurs limites méritent d'être soulignées. D'une part, l'évaluation par le score de silhouette ne capture pas tous les aspects de la qualité d'un clustering : deux partitions de scores similaires peuvent correspondre à des structures très différentes. D'autre part, les hyperparamètres utilisés (ϵ pour DBSCAN, damping pour AP) ont été fixés manuellement ; une approche automatique de sélection serait souhaitable.

Des pistes d'amélioration incluent l'application de méthodes d'optimisation des hyperparamètres (grid search, optimisation bayésienne), l'évaluation sur des jeux de données plus complexes, ainsi que l'intégration de mesures complémentaires telles que Davies-Bouldin et Calinski-Harabasz.

IX. SYNTHÈSE DES CHOIX ALGORITHMIQUES

La sélection d'un algorithme de clustering doit s'appuyer sur une analyse préalable des caractéristiques du problème : connaissance a priori de k , forme supposée des clusters, présence de bruit, et dimension des données. Le tableau suivant synthétise les recommandations issues de cette étude comparative.

Situation / Contrainte	Algorithme recommandé
k connu, clusters sphériques	K-Means
k connu, robustesse aux outliers	K-Medoids
k connu, distribution gaussienne	GMM
k connu, forme quelconque	Spectral Clustering
k inconnu, détection du bruit	DBSCAN / HDBSCAN
k inconnu, densités variables	OPTICS / HDBSCAN
k inconnu, pas de contrainte	Affinity Propagation
Haute dimension + visualisation	UMAP + K-Means
Données catégorielles / mixtes	K-Medoids

Tableau 6 : Guide de sélection algorithmique selon les contraintes du problème

X. CONCLUSION

Ce compte rendu a permis d'explorer de manière systématique onze algorithmes de clustering représentatifs, couvrant les approches partitionnelles, à densité, hiérarchiques, probabilistes et spectrales. L'analyse comparative conduite sur des jeux de données réels de référence confirme qu'il n'existe pas d'algorithme universellement optimal : chaque méthode présente des hypothèses sous-jacentes et des domaines d'application qui lui sont propres.

Les algorithmes partitionnels (K-Means, K-Medoids, K-Medians) offrent un bon compromis entre performance et interprétabilité pour des clusters de forme convexe. Les méthodes à densité (DBSCAN, HDBSCAN, OPTICS) se distinguent par leur capacité à détecter des formes arbitraires et à identifier le bruit. Le clustering hiérarchique et le Spectral Clustering enrichissent l'arsenal pour les structures complexes, tandis que GMM apporte une perspective probabiliste précieuse. Enfin, UMAP combiné à K-Means s'impose comme la référence pour les données de haute dimension.

Cette étude ouvre plusieurs perspectives : l'évaluation sur des données réelles de gestion (segmentation de clientèle, analyse de portefeuille), l'exploration de méthodes hybrides, et l'intégration de ces techniques dans des pipelines d'apprentissage automatique complets. La maîtrise du clustering constitue en effet un prérequis fondamental pour l'analyse exploratoire des données dans tout contexte professionnel ou académique.

BIBLIOGRAPHIE / WEBOGRAPHIE

- [1] Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. KDD, 96(34), 226–231.
- [2] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- [3] McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. arXiv:1802.03426.
- [4] Kohonen, T. (2001). Self-Organizing Maps (3e éd.). Springer.
- [5] Documentation scikit-learn. Clustering. Disponible sur : <https://scikit-learn.org/stable/modules/clustering.html>
- [6] Documentation UMAP. Disponible sur : <https://umap-learn.readthedocs.io/>